DealHunter

Technical Documentation — 1st Delivery

Subject: Web Project

Professors: Roberto Garcia and David Sarrat

Year: 2025 / 2026

Repository: https://github.com/DealHunterOrgan/DealHunter

Members: Otto Biel Ródenas, Sergi Arbós Nicolau, Marc Verdugo Soria, Bruno Verdugo Soria, Joel Marcet Cruz, Marc Prats

1. Application Description

DealHunter is a web application developed with Django that allows users to find the best video game deals across various platforms and digital stores. The main objective is to centralize price and availability information for games in a single place, simplifying purchasing decisions.

The application integrates two external APIs to offer broad coverage:

- CheapShark API: specialized in Steam and other PC store deals. Provides current prices and direct links to offers.
- RAWG API: a video game database that provides detailed information about games, screenshots, trailers, and general data. This API allows for a much more complete catalog than CheapShark alone would offer.

The main features of the application include:

- Browse and search the game catalog with filters by genre, platform, and price range, as well as sorting by price, score, and release date.
- View current prices recorded for each game at each store.
- Direct access to the corresponding store to complete the purchase.
- Game detail page with screenshots, trailer, and user reviews.
- Review system: authenticated users can add, edit, and delete reviews with ratings.
- Management of a personalized Wishlist for authenticated users.
- User registration, authentication, and profile management including avatar selection and password change.
- Django administration interface to manage all model entities.

2. Data Model

2.1 Main Entities

Game

Stores the main information about each video game obtained from the external APIs.

Field	Type	Description
id (PK)	AutoField	Internal unique identifier
title	CharField	Game title
api_ID	CharField (unique)	Identifier in the CheapShark API
steam_appid	CharField (nullable)	Steam ID to query the RAWG API
score	DecimalField	Metacritic score (0–100)
launch_date	DateField (nullable)	Release date
cover_url	URLField	URL of the cover image
genres	ManyToManyField → Genre	Associated genres
platforms	ManyToManyField → Platform	Associated platforms

Shop

Represents the digital stores (Steam, Epic Games, GOG, etc.) where games can be purchased.

Field	Type	Description
id (PK)	AutoField	Unique identifier
name	CharField	Store name
api_ID	CharField (unique)	Identifier in CheapShark
service_state	BooleanField	Indicates whether the store is active
logo_url	URLField	URL of the store logo

Availability

Intermediate table between Game and Shop that records the availability and prices (current and minimum) of a game in a specific store. This entity is the core of the data model.

Field	Type	Description
id (PK)	AutoField	Unique identifier
game (FK)	ForeignKey → Game	Referenced game
shop (FK)	ForeignKey → Shop	Store where it is available
current_price	DecimalField	Current offer price
hist_min_price	DecimalField	Regular price without discount (to calculate savings)
offer_url	URLField	Direct link to the offer

Genre

Represents game genres (Action, RPG, Strategy...), obtained from the RAWG API and the Steam API.

Field	Type	Description
id (PK)	AutoField	Unique identifier
name	CharField (unique)	Genre name

Platform

Represents gaming platforms (Steam, GOG, Epic Games, etc.), inferred from the store of origin of each offer.

Field	Type	Description
id (PK)	AutoField	Unique identifier
name	CharField	Platform name

Wishlist

Intermediate table that links authenticated users (Django User) with games, allowing each user to save games they are interested in.

Field	Type	Description
id (PK)	AutoField	Unique identifier
user (FK)	ForeignKey → User	Wishlist owner user
game (FK)	ForeignKey → Game	Game added to the list
desired_price	DecimalField	User's target price

Review

Allows authenticated users to leave reviews with text and ratings for each game.

Field	Type	Description
id (PK)	AutoField	Unique identifier
game (FK)	ForeignKey → Game	Referenced game
user (FK)	ForeignKey → User	Review author
content	TextField	Review text (max. 1000 characters)
rating	IntegerField	Rating from 1 to 5 (default 5)
created_at	DateTimeField	Creation date (auto)

Profile

Extended user profile, automatically created upon registration, that stores visual preferences.

Field	Type	Description
id (PK)	AutoField	Unique identifier
user	OneToOneField → User	Associated user
avatar_url	CharField	Name of the selected avatar file

2.2 Entity Relationships

The logical relationship diagram is as follows:

- Game ↔ Shop: Many-to-Many relationship through Availability (with price and offer URL attributes).
- Game ↔ Genre: Direct Many-to-Many relationship (a game can have multiple genres).
- Game ↔ Platform: Direct Many-to-Many relationship (a game can be on multiple platforms).
- User ↔ Game: Many-to-Many relationship through Wishlist (with target price).
- User ↔ Game: Many-to-Many relationship through Review (with rating and text).

- User ↔ Profile: One-to-One relationship (each user has exactly one profile).

2.3 Data Model Design Decisions

Use of two complementary APIs: CheapShark provides real-time pricing data but a limited catalog mainly for Steam. RAWG complements the model with more information.

Due to the inability to access games from other platforms (PS5, Nintendo, Xbox...) through the APIs, it was decided that the Platforms table would refer to the website that sells each game (Steam, GOG...).

The desired price monitoring functionality for the user's selected games has not yet been implemented, but is planned for future deliveries.

Intermediate tables with their own attributes: both Availability and Wishlist and Review are not simple relationship tables but contain relevant attributes (prices, ratings, dates). This justifies their treatment as independent entities in the Django model.

Use of Django's built-in User model: for Wishlist and Reviews, the standard Django user model was chosen instead of creating a custom model. The Profile model is associated via OneToOneField to add additional data without modifying the base model.

Uniqueness constraint: Availability has a unique_together constraint on (game, shop) to avoid duplicates. Wishlist has the same constraint on (user, game).

3. Architecture and Project Structure

The project follows the standard Django structure, organized into two main modules:

- core/: main project configuration module. Contains settings.py, urls.py, wsgi.py and asgi.py. Sensitive configuration (SECRET_KEY, DEBUG, RAWG_API_KEY) is loaded from a .env file using python-dotenv.
- games/: main Django application containing models, views, URLs, templates, static files, and data loading scripts.

Data loading is orchestrated through two mechanisms:

- management/commands/load_deals.py: Django command (python manage.py load_deals) that downloads games and deals from CheapShark in a paginated manner (8 pages × 60 games).
- services.py: services module that encapsulates calls to RAWG (multimedia) and CheapShark (alternative price data) APIs, keeping business logic separate from views.

4. Adherence to 12-Factor App Guidelines

The application has been designed with the 12-Factor App methodology in mind.

#	Factor	Description & Compliance
I	**Codebase**	✅ All code is managed in a single public Git repository on GitHub. There are no code forks between environments.
II	**Dependencies**	✅ All dependencies are explicitly declared in requirements.txt and isolated within the Docker container.

III	**Config**	✅ Sensitive configuration (SECRET_KEY, DEBUG, RAWG_API_KEY) is stored in a .env file ignored by version control. settings.py reads it with os.getenv().
IV	**Backing Services**	✅ The database and external APIs (CheapShark, RAWG) are treated as configurable attached resources. Changing the database only requires modifying the configuration.
V	**Build, Release, Run**	✅ Docker strictly separates the image build phase from container execution (run). docker-compose manages the complete lifecycle.
VI	**Processes**	✅ The Django application runs as a stateless process. All persistent data is saved to the database.
VII	**Port Binding**	✅ The application is completely self-contained and exports HTTP by binding to port 8000 through Gunicorn, without depending on an external web server.
VIII	**Concurrency**	✅ All models persist state to the database, with no in-memory variables. Multiple Docker instances can run in parallel without conflicts.
IX	**Disposability**	✅ The stateless nature of the application and the use of Docker containers ensure fast startup and clean shutdown.
X	**Dev/Prod Parity**	✅ The code exclusively uses the standard Django ORM, without any SQLite-specific syntax. The same codebase works identically on any supported database engine.
XI	**Logs**	✅ The application does not manage log files. Django writes events directly to stdout, which Docker collects and manages.
XII	**Admin Processes**	✅ Administrative tasks (migrations, data loading) are executed as one-off processes in the same environment as the application, via manage.py.

5. Authentication and User Management

The application implements a complete system for user authentication, registration, and profile management:

- Authentication: a custom view (CustomLoginView) extending Django's LoginView is used.
- Registration: a custom registration view (SignUpView) using CustomUserCreationForm has been implemented, adding the email field. Upon completing registration, the user is automatically authenticated and their associated Profile is created.
- Logout: managed by Django's built-in view with redirection to the main page.
- View protection: features requiring authentication (wishlist, profile, reviews, account deletion) are protected with LoginRequiredMixin, redirecting unauthenticated users to the login page.
- User profile: ProfileView displaying the wishlist, latest reviews, total accumulated savings, and Hunter level.
- Account editing: EditAccountView allowing modification of username, password, and avatar. Password change keeps the session active thanks to update_session_auth_hash.

- Account deletion: DeleteAccountView with explicit confirmation before permanent deletion.
- Redirects: LOGIN_REDIRECT_URL and LOGOUT_REDIRECT_URL are configured in settings.py to direct the user to the main page.

6. Containerization with Docker

The application includes all necessary files to run as a Docker container orchestration:

- Dockerfile: defines the application image and installs Python dependencies. Also configures Gunicorn as the production WSGI server.
- docker-compose.yml: orchestrates the web service and maps port 8000. Mounts the necessary volumes for the SQLite database and static files, and automatically runs migrations and static file collection on startup.

The container startup sequence is: migrate → collectstatic → gunicorn. This ensures the database is always up to date and static files are available before the application starts accepting requests.

7. User Interface and Navigation

- Game catalog (Home): main view with all catalog games paginated in groups of 30. Includes filters by genre, platform, and price range. Also includes sorting options by price (ascending/descending), Metacritic score, and release date. The sidebar shows the count of games per category and platform.
- Game detail: individual game view with cover, star rating, description from RAWG, genres, platforms, expandable screenshots, and video trailer. Shows the current price and direct link to the store. Includes the comments and ratings system.
- User profile: personalized page for authenticated users with animated avatar, Hunter level, statistics (games in wishlist, reviews written, total savings), the complete wishlist, and the latest reviews made.
- Account editing: form to modify username, password, and avatar (6 options available).
- Account deletion: confirmation page before permanent deletion of the account and all associated data.
- Login / Registration: access and account creation forms with password validation.

8. Deployment Instructions

8.1 Option A — Local Execution

1. 1. Clone the repository: git clone https://github.com/DealHunterOrgan/DealHunter.git
2. 2. Navigate to the directory: cd DealHunter
3. 3. Create and activate the virtual environment: python -m venv dhunter && source dhunter/bin/activate
4. 4. Install dependencies: pip install -r requirements.txt
5. 5. Create the .env file at the root with the variables: SECRET_KEY, RAWG_API_KEY
6. 6. Apply migrations: python manage.py migrate
7. 7. Load data: python manage.py load_deals

8. 8. Collect static files: python manage.py collectstatic
9. 9. Start the server: python manage.py runserver
10. 10. Access the application at: http://localhost:8000

8.2 Option B — Deployment with Docker

11. 1. Ensure Docker and Docker Compose are installed.
12. 2. Create the .env file at the project root with the required variables.
13. 3. In the project root directory, run: docker-compose up --build
14. 4. Access the application at: http://localhost:8000
15. 5. To stop the containers: docker-compose down

8.3 Option C — Public Access

The application is publicly accessible via the following URL:

https://dealhunter-e1zt.onrender.com/